King's College London · 本科毕业设计（Final Year Project）

在本地小模型上增强多智能体 Web 代理的可靠性

面向 Magentic-UI 的弱校验与按需模型路由

Enhancing the Reliability of a Local-LLM Multi-Agent Web Agent: Weak Validation and On-Demand Model Routing for Magentic-UI

作者（Author）	Haolun Shan

课题阶段	Stage 5 — 设计与实现报告
技术栈	Magentic-UI • qwen2.5:14b • Ollama • Docker (Windows)
日期	2026-06-26

目录

摘要 Abstract

1 引言 Introduction

2 背景与相关工作 Background & Related Work

3 系统设计 System Design

4 实现 Implementation

5 评估 Evaluation

6 局限与未来工作 Limitations & Future Work

7 结论 Conclusion

参考文献 References

摘要

Magentic-UI 是微软 Magentic-One 多智能体框架的开源实现:一个中央 Orchestrator(协调器)负责把任务拆成计划、逐步评估进度,并把具体的网页操作派给 WebSurfer 等执行型智能体。该框架在设计上默认底层语言模型的输出"总是合法的"——计划与进度账本(ledger)永远是结构正确的 JSON、工具调用参数永远符合执行器的签名。然而当我们把后端从云端大模型替换为**本地小模型**(qwen2.5:14b,经 Ollama 在单张 16GB 显卡上运行)时,这个假设被频繁打破:小模型会吐出畸形 JSON、字段名错位、把参数包进数组、甚至退化成自然语言伪代码,导致执行器崩溃或代理陷入空转。

本报告提出并实现了一个建立在 Magentic-UI 之上的**可靠性增强层**,核心思想是**纵深防御**与"先免费确定性、后付费随机性"。该层由五个机制组成:(1) **弱校验器**,以纯规则方式把模型输出归一化为标准形,给出 PASS / REPAIR / ESCALATE 三态判决;(2) **执行层补救**,在工具调用真正执行失败时做确定性重试与下拉项兜底;(3) **按需模型路由**,只有当本地模型持续卡住时才临时借用云端强模型,且区分"一次性"与"接管整段"两种模式;(4)(5) 云端 **JSON Rescue** 与 **Web Text Rescue**,作为最后的格式与文本兜底。前两层完全本地、零成本、可复现,只有少数真正棘手的情形才升级到云端。

在开发期的代表性运行中,绝大多数模型输出缺陷被前两层免费拦截并修复;只有持续无进展的少数任务才触发云端路由,单任务云端开销约在一美分量级。我们同时识别出一个尚未解决的盲区——"执行了但没有真正进展"的循环(如死元素/下拉框反复操作),并据此提出一个元素级 loop-breaker 作为未来工作。本工作表明,通过一层薄而确定的校验与路由,可以让本地小模型在原本为云端大模型设计的多智能体框架中稳定运行,同时把云端成本压到最低。

1 引言

1.1 背景与动机

基于大语言模型(LLM)的智能体近年来被用于自动化复杂的网页任务:阅读页面、点击、填表、检索信息。微软提出的 Magentic-One 是其中具有代表性的**多智能体**架构,它用一个 Orchestrator 统一规划与调度,把浏览器操作交给专门的 WebSurfer 智能体。其开源实现 Magentic-UI 提供了完整的可运行系统。

这类系统通常假设后端是 GPT-4 级别的云端大模型。然而出于**成本、隐私与可离线运行**的考虑,把它跑在**本地小模型**上具有现实意义:本课题在一台配备单张 16GB 显存显卡的 Windows 机器上,通过 Ollama 运行 qwen2.5:14b,完全本地、零 API 费用地驱动 Magentic-UI。

1.2 问题陈述

把云端大模型替换为本地小模型后,系统的**瓶颈从"能力"转移到了"可靠性"**。qwen2.5:14b 在语义上往往"想对了",但在**输出格式**上频繁出错。我们在实跑日志中观察到的典型缺陷包括:

- 进度/计划账本本应是结构化 JSON,却出现字段缺失、布尔值写成字符串、整条对象被包进数组;
- 工具调用参数错位:元素 id 本应是整数,却写成 `{"id": 29}` 这样的嵌套对象、字符串、甚至一句自然语言描述;
- 字段名漂移:同一个"目标元素"被叫成 target_id / id / element_id / boundingBoxId 等等;
- 模型把多个动作打包进一次调用,或干脆停止发原生工具调用、改用伪代码描述操作;
- 长任务下 Ollama 默认 4096 的上下文窗口把历史**截断**,模型"半盲"地决策。

原版 Magentic-UI 对这些缺陷没有防护:畸形输出被直接送入执行器,轻则该步无效、重则抛异常,代理随后陷入"卡住→重规划→再卡住"的循环。直接的修复思路是"出问题就调云端大模型",但这会牺牲掉本地部署的全部成本与隐私优势。

1.3 目标与贡献

本工作的目标是:**在尽量不修改 Magentic-UI 核心文件、且尽量不产生云端费用的前提下,显著提升本地小模型驱动下的任务可靠性**。主要贡献如下:

- **一个三态弱校验器**:用纯标准库规则把模型输出归一化为标准形,以 PASS / REPAIR / ESCALATE 三态做确定性判决,零模型调用、可单元测试;
- **一个执行层确定性补救机制**:在工具调用真正抛异常时利用实时页面状态做重试与下拉项兜底;
- **一个按需模型路由机制**:仅在持续无进展时临时借用云端强模型,区分"一次性"与"接管整段(sticky)"两种策略,并把云端调用控制为文本-only 以压低成本;
- **两个云端兜底**(JSON Rescue / Web Text Rescue)作为最后防线;
- **配套工程**:统一的 MODEL_TRACE 日志、token 与成本统计工具、自动续算重规划,以及把 Ollama 上下文从 4096 解锁到 16384 的部署调整;
- 对一个尚未解决的**盲区**("执行了但没进展"循环)的分析,并提出元素级 loop-breaker 的未来方案。

1.4 报告结构

第 2 章介绍 Magentic-One/Magentic-UI 的架构与本地小模型带来的缺口;第 3 章给出整体设计原则与五机制总览;第 4 章详述各机制的实现;第 5 章给出开发期评估与成本分析;第 6 章讨论局限与未来工作;第 7 章总结。

2 背景与相关工作

2.1 Magentic-One 与 Magentic-UI

Magentic-One(Fourney
等,微软,arXiv:2411.04468)是一个面向复杂任务的通用多智能体系统。它的核心是一个 **Orchestrator(协调器)**:它本身不直接操作环境,而是负责**把任务拆成计划、评估每一步的进度、决定下一步派给哪个智能体**。执行型智能体中,WebSurfer
负责真实的浏览器操作(搜索、点击、输入、滚动、读取页面)。Magentic-UI
是该思想的开源、可运行实现,本工作即在其上展开。

2.2 两个账本:Task Ledger 与 Progress Ledger

Orchestrator 通过两个"账本"驱动整个任务,这是理解本工作插入点的关键。

Task Ledger(任务账本)= 计划。 任务开始时,Orchestrator 收集已知/待查/待推的事实,并产出一份分步计划(Step 1 谁做、Step 2 谁做……)。这是**外层循环**;一旦卡住,就回到这里**重写计划(replan)**。

Progress Ledger(进度账本)= 每一步的体检表。 每走一步,Orchestrator 让模型输出一段 JSON,逐项判断:任务是否已满足(is_request_satisfied)、是否在原地打转(is_in_loop)、当前步是否完成(is_current_step_complete)、下一步该谁说话(next_speaker)、给它什么指令。这是**内层循环**。

2.3 stall → replan 循环

两个账本共同构成一个反馈环:内层不断填写进度账本;当进度账本判定"没有进展"或"在打转"时,Orchestrator 累计 stall 信号,达到阈值便触发 replan,回到外层重写计划。**整套机制默认模型产出的账本 JSON 与工具调用参数永远合法。**本工作正是补在这一假设失效之处。

2.4 Set-of-Marks 与元素编号

WebSurfer 借助 Set-of-Marks 思路:给页面上每个可交互元素叠加一个**数字标号**,模型只需引用编号(如"点击 6 号")即可操作。编号到真实 DOM 元素的映射(element_id_mapping / rects)由 WebSurfer 维护。需要强调:本工作的弱校验器只规整模型给出的**编号格式**,并不负责"编号→真实元素"的映射。

2.5 本地小模型带来的可靠性缺口

qwen2.5:14b(Qwen 团队,阿里巴巴)是一个非视觉的开源模型,本工作经 Ollama 在 Docker 中以纯文本方式驱动 WebSurfer(不发送截图,从而避免视觉 token 带来的开销与限流)。它在格式遵循上的不稳定,正是第 1.2 节列出的各类缺陷的根源。下文的增强层即针对性地填补这些缺口。

3 系统设计

3.1 设计原则:纵深防御,先免费后付费

整层遵循两条原则。其一是**纵深防御**:一个缺陷会依次经过多道防线,任何一道能处理就地解决,处理不了再交给下一道。其二是**先免费确定性、后付费随机性**:靠前的层是纯规则、零成本、可复现的本地处理;只有当所有免费层都无能为力、且系统确实持续无进展时,才升级到**花钱、且不确定**的云端模型。这样把昂贵的云端调用留给真正值得的少数情形。

3.2 五个机制总览

机制	层次/时机	是否调云	成本	处理什么
弱校验器 Weak Validator	执行/规划**前**	否 • 确定性	免费	畸形 JSON、参数错位、字段名漂移;三态判决
执行补救 Exec Rescue	工具执行**崩溃后**	否 • 确定性	免费	别名重试、表单提交、下拉项 select_option 兜底
模型路由 Model Routing	持续无进展时	是 • 决策	~$0.01/任务	借云端强模型重新规划/决策
JSON 兜底 JSON Rescue	校验器 ESCALATE 后	是 • 格式	~少量	云端把畸形计划 JSON 重修为合法格式
文本兜底 Web Text Rescue	动作说明文本缺失	是 • 文本	~$0(常跳过)	改写展示用文本(对执行无影响,常被本地默认值替代)

表 3-1 可靠性增强层的五个机制。前两层完全本地、零成本;后三层仅在必要时触达云端。

3.3 在 Magentic-One 架构上的插入点

下表把每个机制对应到 Magentic-One 的具体环节,说明它们"补在哪个接缝上"。

Magentic-One 环节	本地小模型会出的问题	本层的应对
Orchestrator 产出 计划/进度账本 JSON	畸形 JSON(字段名错、被包成数组)	弱校验器三态;修不了再 JSON Rescue 上云
进度账本判定"无进展"→ replan	本地重规划仍卡住	Model Routing 借云端强模型重新决策
进度账本的 is_in_loop / 进度判断	"执行了但没进展"判不出(死元素循环)	**当前盲区**;见第 6 章 loop-breaker
next_speaker 派给 WebSurfer 执行	反复点死元素 / 工具调用参数错位	弱校验器规整参数 + Exec Rescue 补救

表 3-2 增强层与 Magentic-One 架构的对应关系。

4 实现

4.1 弱校验器(Weak Validator)

弱校验器(_weak_validator.py)是整层的第一道防线,**纯标准库实现、不调用任何模型**。它的核心理念是:不去枚举"所有可能的错法",而是**尝试把模型输出重建成唯一的标准形,重建的结果本身就是判决**。

4.1.1 三态与统一返回契约

所有校验函数都返回同一个三元组 `(state, args, reason)`:

- `state` —— 判决:PASS / REPAIR / ESCALATE;
- `args` —— 要执行的标准化参数 dict(PASS/REPAIR);ESCALATE 时原样返回;
- `reason` —— 给日志看的人类可读原因。

统一形状让上层用一段代码即可处理三种结局,与具体工具无关——这是把弱校验做成可插拔一层的关键:

```
state, args, reason = validate_tool_call(name, raw_args)
print(f"[WEAK_VALIDATOR] {state} | tool={name} | {reason}")
if state != "ESCALATE":
    tool_args = args        # PASS/REPAIR use canonical args
# ESCALATE: leave it for downstream Exec Rescue
```

4.1.2 归一化即判决

以点击校验 _validate_click 为例,它做三件事:(1) 用**别名表** _first_alias 找出"哪个键装着目标 id"(target_id / target / id / element_id / boundingBoxId ……);(2) 用 _coerce_id 把取出的值**掰成整数**——整数原样、数字字符串转整数、嵌套对象 `{"id":29}` 钻进去取、列表 `[17,21]` 取第一个、布尔值故意拒绝;(3) 拼出标准形 canonical 后:

```
state = PASS if canonical == args else REPAIR
```

判决就落在这一行 `dict` 相等比较上:与标准形一字不差即 PASS(本来就对),不同即 REPAIR(我修过了);若某个必需字段 _coerce_id 返回 None(掰不动),则在拼装前就 ESCALATE。换言之,"好"被定义为"等于我会构造的那个标准形",校验器对质量没有主观判断,只有一个标准答案和一次相等比较。

4.1.3 三态实例

输入(模型吐出的 target_id)	_coerce_id 结果	判决
6	6(本来就是整数)	PASS
"29" / {"id":29} / [17,21]	29 / 29 / 17	REPAIR
"the blue button" / true / 字段缺失	None(掰不动)	ESCALATE

表 4-1 同一条点击调用,target_id 取不同值时的三态判决。

4.1.4 三扇门

弱校验器对外有三个入口,分别处理不同形态的输入:

入口	处理对象	关键内部方法

validate_tool_call	WebSurfer 工具调用(click/input_text/web_search)	_unwrap_action_array → _validate_* → _first_alias → _coerce_id
validate_plan_output / weak_rescue_json	Orchestrator 的计划/进度账本 JSON	_coerce_bool / _coerce_id;ESCALATE 后交云端 JSON Rescue
recover_tool_call_from_text	模型连 JSON 都不吐、退化成伪代码时	_iter_json_objects / _parse_ibaction(**此处才用正则**)

表 4-2 弱校验器的三个入口。判错与修复主要靠结构与类型规则;正则仅用于最后的文本恢复。

需要澄清一个常见误解:弱校验器判错**主要不靠正则**,而是对解析出的对象做结构与类型归一化;只有当模型彻底跑偏、吐出 `/IBAction: click(17)` 这类伪代码、连 JSON 都解析不出来时,才退到正则从纯文本里捞出工具调用。

4.2 执行层补救(Exec Rescue)

执行补救(_web_execution_rescue.py 的 try_execution_rescue)是弱校验器的下一道防线,同样**确定性、零成本**,但工作时机不同:它在**工具调用真正执行抛异常之后**(或被弱校验器 ESCALATE 而主动 raise 之后)才介入。它与弱校验器的本质区别在于——**它能看到实时页面状态**(rects / element_id_mapping),而弱校验器只能看到参数字典。

它依次尝试三件事:

- **归一化重试**(_normalize_execution_call):更"执行口味"的别名修复,如把内嵌的 tool 名拎出来、id→target_id、value/text→text_value,改完重试一次;
- **表单启发式**:输入框名字像 from/to/location/destination/address/station 时自动 press_enter,让"输入地点"真正提交;
- **下拉项兜底**(select_option):当一次 input_text 失败、且页面此刻出现 role=option 的元素(autocomplete 下拉展开)时,改发 select_option 选中第一个选项。

都不奏效则记录 `[WEB_EXEC_RESCUE] gave up` 并返回 None,该动作报失败。因为它是一串固定 `if/else` 且"选第一个 option"这类规则写死、不调模型,所以它是确定性的——同样的(失败动作 + 页面状态)永远给出同样的补救,可复现、可单测。

4.3 按需模型路由(Model Routing)

当本地确定性层都救不动、系统仍持续无进展时,才进入唯一会主动花钱的机制:模型路由(_model_rotation.py)。它**临时借用云端强模型**(Orchestrator 用 gpt-4o,WebSurfer 用 gpt-4o-mini),卡点过去后再交还本地。

4.3.1 三种触发

触发	信号来源	默认阈值(环境变量)
stall(卡住)	进度账本 need_to_replan=true 累计	ROUTE_AFTER_STUCK = 2
idle(空转)	连续若干轮 is_current_step_complete≠true	ROUTE_AFTER_IDLE = 3

no-op(空回合)	WebSurfer 一回合没发任何工具调用	WEBSURFER_ROUTE_AFTER_NOOPS = 1

表 4-3 模型路由的三种触发条件及其阈值开关。

4.3.2 一次性 vs. 接管整段(sticky)

路由有两种"借用时长"策略,这是成本与效果之间的核心权衡:

ONE-SHOT(默认)	STICKY(接管整段)
借一次就立刻交还本地。 最便宜、尽量留在本地。 但强模型只看一眼,可能浅。 ~1 次云调用 · ~$0.01/任务	一旦升级,强模型开完后续整段 run。 难任务、多步导航更稳。 文本-only 历史使其仍便宜。 云端驱动 · ~$0.05/任务

表 4-4 一次性与 sticky 两种路由策略,由 WEBSURFER_ROUTE_STICKY 开关切换。

当前实现中,**sticky 仅对 WebSurfer 有效**;Orchestrator 一侧只有一次性。设计理由是:WebSurfer 每一步都要操作,一旦"手"卡住,后续几十步大概率继续卡,让强模型接管整段更划算;而 Orchestrator 每次只做一个高层决策,借完即还更合理。

4.3.3 成本控制

两点把云端开销压到最低:其一,WebSurfer 全程**文本-only**(qwen 非视觉,本就不发截图),因此即便升级到 gpt-4o-mini 也不产生视觉 token,避免了 TPM 限流;其二,对强模型调用设**消息条数上限**(ROUTE_MSG_CAP),只把最近的关键上下文交给云端,而非整段历史。

4.4 云端兜底:JSON Rescue 与 Web Text Rescue

两个云端兜底处于防线末端。**JSON Rescue**(_json_rescue.py)接收弱校验器对计划/账本 JSON 判 ESCALATE 的情形,用云端模型把畸形 JSON 重修为合法格式。**Web Text Rescue**(web_surfer/_web_rescue.py)只改写**展示用文本**(动作说明、最终答复措辞),对浏览器执行没有影响;且其中"动作说明"一类已被一个零 token 的本地默认值兜底替代——付费为展示文本生成内容并不改善执行,反而会主导 token 消耗,故被刻意省去。

4.5 工程化支撑

- **统一追踪日志**:所有模型调用打印 [MODEL_TRACE] 行并镜像到文件(TRACE_LOG),记录本地/云端、模型名、token;
- **token 与成本统计**(token_stats.py):按 run 切分日志,汇总本地 vs 云端 token 与折算费用;
- **自动续算重规划**(AUTO_REPLAN):中途重规划不再阻塞等待用户"接受计划",新计划仍展示但不打断执行;
- **通俗化命名**:环境变量与函数统一为 ROUTE_* / WEBSURFER_ROUTE_* 等可读名称,便于配置与讲解。

4.6 部署:解锁 Ollama 上下文窗口

一个关键的部署发现:Ollama 默认上下文长度仅 4096,长任务下历史被截断,模型"半盲"决策——这并非真实历史过长(实测可达数万 token),而是 num_ctx 截断。把容器的 OLLAMA_CONTEXT_LENGTH 提升到 **16384** 后(在 16GB 显存上经权衡选择,未取 32K),qwen 的有效 prompt token 明显回升,卡顿显著缓解。为规避主流搜索引擎的验证码/同意墙,WebSurfer 的默认搜索改为 DuckDuckGo 的无 JS 端点(html.duckduckgo.com/html)。

4.7 工程约束:最小化核心文件改动

为便于与上游 Magentic-UI 对齐、降低维护成本,本工作把逻辑尽量放在**独立新增模块**中,核心文件(如 _orchestrator.py / _web_surfer.py)只做**导入与调用**级别的改动,并以注释 HAOLUN PATCH 明确标注,便于审阅与回滚。

5 评估

本章给出开发期的观察性评估。**需要说明:以下数据来自开发与调试阶段的代表性运行,用于说明各机制的行为与量级,尚不构成受控基准实验;系统化的、跨多站点多任务的定量评估列为未来工作。**

5.1 实验环境

项目	配置
本地模型	qwen2.5:14b(非视觉),经 Ollama 运行
硬件	单张 16GB 显存显卡,Windows 11
上下文长度	16384(由默认 4096 提升)
云端强模型	Orchestrator: gpt-4o;WebSurfer: gpt-4o-mini
检索端点	html.duckduckgo.com/html(无 JS)
WebSurfer 模式	纯文本(不发送截图)

表 5-1 评估环境配置。

5.2 弱校验器三态分布

在一次代表性运行(开发记录 run 84,任务为定向检索某研究实验室的近期论文)中,本地模型产出的多数工具调用都带有可被本地修复的格式缺陷,弱校验器将其逐一 RECOVER/REPAIR,**该次运行 100% 本地完成、未触达任何云端调用、费用为 $0**。这印证了"绝大多数缺陷在前两层即被免费解决"的设计预期。

【建议补充实测】用 token_stats.py 汇总若干次运行的 PASS / REPAIR / ESCALATE 条数占比,做成一张分布柱状图或表,以量化"多数缺陷被本地免费修复"的结论。

5.3 一次性 vs. sticky 的成本-效果对比

在同一类需要多步导航的难任务上,我们对比了两种路由策略的代表性运行:

策略	代表运行	云端调用	约成本	效果
one-shot	run 86	约 1 次	~$0.012	便宜,但强模型只看一眼,抓到偏题结果
sticky	run 80 / 84	整段接管	~$0.05	由强模型驱动完成了较难的导航

表 5-2 两种路由策略在难任务上的代表性对比:one-shot 更省但偏浅,sticky 更稳但稍贵。

结论方向:对简单卡点,一次性已足够;对多步、强依赖连续操作的难任务,sticky 的额外成本换来明显更高的完成率,值得。两者均因文本-only 而维持在分量级开销。

5.4 上下文长度的影响

把 Ollama 上下文从 4096 提升到 16384 后,qwen 的有效 prompt token 明显回升(开发观察中升至约一万量级),此前因截断导致的"莫名回到上一步""反复卡顿"显著缓解。这表明:在评判本地小模型"能力不足"之前,必须先排除"上下文被截断"这一部署层面的假性缺陷。

【建议补充实测】对同一任务分别在 4096 与 16384 下各跑若干次,记录平均步数、卡顿次数、是否完成,做一张前后对比表,以量化上下文解锁的收益。

5.5 成本分析

整层的成本结构与设计意图一致:弱校验器与 Exec Rescue **零边际成本**;Web Text Rescue 的动作说明一类被本地默认值替代后基本不产生费用;真正的开销集中在少数触发模型路由的任务,且因文本-only与消息上限而维持在**单任务一美分到数美分**量级。换言之,大量缺陷被免费层吸收,云端预算被定向投放到"值得救"的少数硬卡点上。

6 局限与未来工作

6.1 "执行了但没进展"的盲区

当前所有确定性层都只能捕捉"**崩溃**"(抛异常),而捕捉不了"**执行成功但没有真正进展**"。最典型的是死元素 / 下拉框循环:模型反复对一个不生效的元素发出格式完全合法的点击,执行器不报错、页面也确实"抖一下"(下拉展开),于是这类"有效的废动作"从所有现有的网里漏了下去。

检测层	为何抓不到下拉框循环
弱校验器	点击格式完全合法(target_id 是正整数)→ 判 PASS,无可修
Exec Rescue	_execute_tool 不抛异常(点击"成功"了)→ 触发不了
idle 空转检测	依赖进度账本 is_current_step_complete;弱判官把"页面抖一下"误当进展,把 idle 计数清零 → 不触发;即便触发也只换 Orchestrator 大脑,而非 WebSurfer 这只"手"
no-op 计数	下拉框循环每轮都在发点击 → 计数永不增长

表 6-1 下拉框循环为何同时逃过现有四道检测——它既不崩溃、也不空闲,而是"持续做有效的废动作"。

6.2 为何 idle 解决不了下拉框

尤其需要说明 idle 的失效:idle 不自己看页面,它读取的 is_current_step_complete **同样由卡住的 qwen 填写**。下拉框每次点击都让页面发生变化,弱判官容易把"有东西动了"误判成"这步推进了",于是 idle 计数被反复清零,永远到不了阈值。退一步说,即便 idle 触发,它换的是**规划脑**(Orchestrator),但下拉框的错不在计划,而在 WebSurfer 的**执行机械动作**——换脑不改变手的循环。

6.3 未来工作:元素级 loop-breaker

解决之道是一个放在 WebSurfer 执行层、**不依赖进度判官主观意见**的元素级循环检测器:记录"同一个 target_id / 同一动作被连续发出 N 次"或"动作后 DOM 状态实质未变",一旦命中就**强制换招**(改用 select_option、滚动,或直接路由 WebSurfer),从而打断机械循环。这与 idle 的"数空转轮数"在粒度和所在层次上都不同,是对当前盲区的针对性补全。

6.4 其它局限

- 评估尚属观察性,缺乏跨多站点、多任务的受控基准与统计显著性;
- 弱校验器目前覆盖 click / input_text / web_search 三类高频工具,其余工具按 PASS 放行,未来可扩展;
- 路由阈值与 sticky 策略为经验设定,可进一步做成自适应;
- 对单一本地模型(qwen2.5:14b)调优,跨模型/跨规模的普适性有待验证。

7 结论

本工作在 Magentic-UI 之上构建了一层薄而确定的**可靠性增强层**,使原本为云端大模型设计的多智能体 Web 代理能够稳定地运行在本地小模型(qwen2.5:14b)之上。其核心是**纵深防御**与**先免费确定性、后付费随机性**:弱校验器以"归一化即判决"的方式用纯规则把畸形输出修回标准形(PASS/REPAIR/ESCALATE);Exec Rescue 在执行崩溃后利用实时页面状态做确定性补救;只有当本地层确实救不动、系统持续无进展时,模型路由才临时借用云端强模型,并以一次性/接管整段两种策略在成本与效果间取舍。

开发期观察表明:绝大多数本地模型的格式缺陷被前两层免费拦截并修复,云端预算被定向投放到少数真正棘手的卡点,单任务开销维持在分量级。我们同时清晰地界定了一个尚未解决的盲区——"执行了但没进展"的循环,并据此提出元素级 loop-breaker 作为下一步工作。总体而言,本工作给出了一条把昂贵的云端多智能体能力"下放"到本地小模型的可行、低成本路径。

参考文献

[1] Fourney, A., et al. (2024). Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks. Microsoft Research. arXiv:2411.04468.

[2] Microsoft. (2025). Magentic-UI (open-source implementation of Magentic-One). GitHub: microsoft/magentic-ui.

[3] Yang, J., et al. (2023). Set-of-Mark Prompting Unleashes Extraordinary Visual Grounding in GPT-4V. arXiv:2310.11441.

[4] Qwen Team, Alibaba. (2024). Qwen2.5 Technical Report. arXiv:2412.15115.

[5] Ollama. (2024). Ollama: Run large language models locally. https://ollama.com.